

1. Alloy Composition Analysis System

Description:

Design a system to analyze alloy compositions using structures for composition details, arrays for storing multiple samples, and unions to represent percentage compositions of different metals.

Specifications:

- Structure: Stores sample ID, name, and composition details.
- Union: Represents variable percentage compositions of metals.
- Array: Stores multiple alloy samples.
- const Pointers: Protect composition details.
- Double Pointers: Manage dynamic allocation of alloy samples.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Alloy
```

```
{
```

```
    const int *sampleID;
```

```
    char name[50];
```

```
    union
```

```
    {
```

```
        float iron;
```

```
        float aluminum;
```

```
        float copper;
```

```
    } composition;
```

```
    int metalType; // 1 for Iron, 2 for Aluminum, 3 for Copper
```

```
};
```

```
void addAlloy(struct Alloy **alloys, int *sampleCount);  
void displayAlloyDetails(struct Alloy *alloys, int sampleCount);
```

```
int main()  
{  
    struct Alloy *alloys = NULL;  
    int sampleCount = 0, option;  
    do  
    {  
        printf("\nAlloy Composition Analysis System\n");  
        printf("1. Add Alloy Sample\n");  
        printf("2. Display Alloy Samples\n");  
        printf("3. Exit\n");  
        printf("Enter your option: ");  
        scanf("%d", &option);  
        switch (option)  
        {  
            case 1: addAlloy(&alloys, &sampleCount);  
                    break;  
            case 2: displayAlloyDetails(alloys, sampleCount);  
                    break;  
            case 3: for (int i = 0; i < sampleCount; i++)  
                        free((int *)alloys[i].sampleID);  
                    free(alloys);  
                    printf("Exiting the system\n");  
                    break;  
            default: printf("Invalid option\n");  
        }  
    }  
}
```

```

    }
} while (option != 3);
return 0;
}

```

```

void addAlloy(struct Alloy **alloys, int *sampleCount)
{
    int tempSampleID;
    struct Alloy newAlloy;
    printf("Enter Sample ID: ");
    scanf("%d", &tempSampleID);
    int *sampleIDPtr = (int *)malloc(sizeof(int));
    *sampleIDPtr = tempSampleID;
    newAlloy.sampleID = sampleIDPtr;
    printf("Enter Alloy Name: ");
    scanf(" %[^\\n]", newAlloy.name);
    printf("Enter Metal Type (1 for Iron, 2 for Aluminum, 3 for Copper): ");
    scanf("%d", &newAlloy.metalType);
    switch (newAlloy.metalType)
    {
        case 1: printf("Enter percentage of Iron: ");
                scanf("%f", &newAlloy.composition.iron);
                break;
        case 2: printf("Enter percentage of Aluminum: ");
                scanf("%f", &newAlloy.composition.aluminum);
                break;
        case 3: printf("Enter percentage of Copper: ");

```

```

        scanf("%f", &newAlloy.composition.copper);
        break;
    default: printf("Invalid metal type\n");
        return;
}
*alloys = (struct Alloy *)realloc(*alloys, (*sampleCount + 1) *
                                   sizeof(struct Alloy));

(*alloys)[*sampleCount] = newAlloy;
(*sampleCount)++;
printf("Alloy sample added successfully\n");
}

void displayAlloyDetails(struct Alloy *alloys, int sampleCount)
{
    if (sampleCount == 0)
    {
        printf("No alloy samples available\n");
        return;
    }
    printf("\nAlloy Sample Details\n");
    for (int i = 0; i < sampleCount; i++)
    {
        printf("Sample ID: %d\n", *(alloys[i].sampleID));
        printf("Alloy Name: %s\n", alloys[i].name);
        switch (alloys[i].metalType)
        {
            case 1: printf("Metal: Iron\n");

```

```

        printf("Percentage: %.2f%%\n", alloys[i].composition.iron);
        break;
    case 2: printf("Metal: Aluminum\n");
        printf("Percentage: %.2f%%\n", alloys[i].composition.aluminum);
        break;
    case 3: printf("Metal: Copper\n");
        printf("Percentage: %.2f%%\n", alloys[i].composition.copper);
        break;
    default: printf("Invalid metal type\n");
}
}
}

```

2. Heat Treatment Process Manager

Description:

Develop a program to manage heat treatment processes for metals using structures for process details, arrays for treatment parameters, and strings for process names.

Specifications:

- Structure: Holds process ID, temperature, duration, and cooling rate.
- Array: Stores treatment parameter sets.
- Strings: Process names.
- const Pointers: Protect process data.
- Double Pointers: Allocate and manage dynamic process data.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct HeatTreatment
```

```
{  
    const int *processID;  
    char processName[50];  
    float temperature;  
    float duration;  
    float coolingRate;  
};
```

```
void addProcess(struct HeatTreatment **processes, int *processCount);
```

```
void displayProcesses(struct HeatTreatment *processes, int processCount);
```

```
int main()
```

```
{  
    struct HeatTreatment *processes = NULL;  
    int processCount = 0, option;  
    do  
    {  
        printf("\nHeat Treatment Process Manager\n");  
        printf("1. Add Process\n");  
        printf("2. Display Processes\n");  
        printf("3. Exit\n");  
        printf("Enter your option: ");  
        scanf("%d", &option);  
        switch (option)
```

```

{
    case 1: addProcess(&processes, &processCount);
        break;
    case 2: displayProcesses(processes, processCount);
        break;
    case 3: for (int i = 0; i < processCount; i++)
        free((int *)processes[i].processID);
        free(processes);
        printf("Exiting\n");
        break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addProcess(struct HeatTreatment **processes, int *processCount)
{
    int tempProcessID;
    struct HeatTreatment newProcess;
    printf("Enter Process ID: ");
    scanf("%d", &tempProcessID);
    int *processIDPtr = (int *)malloc(sizeof(int));
    *processIDPtr = tempProcessID;
    newProcess.processID = processIDPtr;
    printf("Enter Process Name: ");
    scanf(" %[^\n]", newProcess.processName);
}

```

```

printf("Enter Temperature: ");
scanf("%f", &newProcess.temperature);
printf("Enter Duration: ");
scanf("%f", &newProcess.duration);
printf("Enter Cooling Rate: ");
scanf("%f", &newProcess.coolingRate);
*processes = (struct HeatTreatment *)realloc(*processes, (*processCount + 1)
                                           * sizeof(struct HeatTreatment));

(*processes)[*processCount] = newProcess;
(*processCount)++;
printf("Heat treatment process added successfully\n");
}

```

```

void displayProcesses(struct HeatTreatment *processes, int processCount)
{
    if (processCount == 0)
    {
        printf("No heat treatment processes available\n");
        return;
    }
    printf("\nHeat Treatment Process Details:\n");
    for (int i = 0; i < processCount; i++)
    {
        printf("Process ID: %d\n", *(processes[i].processID));
        printf("Process Name: %s\n", processes[i].processName);
        printf("Temperature: %.2f Celsius\n", processes[i].temperature);
        printf("Duration: %.2f hours\n", processes[i].duration);
    }
}

```

```
        printf("Cooling Rate: %.2f Celsius/hour\n", processes[i].coolingRate);
    }
}
```

3. Steel Quality Monitoring

Description:

Create a system to monitor steel quality using structures for test results, arrays for storing test data, and unions for variable quality metrics like tensile strength and hardness.

Specifications:

- Structure: Stores test ID, type, and result.
- Union: Represents tensile strength, hardness, or elongation.
- Array: Test data for multiple samples.
- const Pointers: Protect test IDs.
- Double Pointers: Manage dynamic test records.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct SteelTest
{
    const int *testID;
    char testType[50];
    union
    {
        float tensileStrength;
```

```
    float hardness;
    float elongation;
} result;
int metricType; // 1 for Tensile Strength, 2 for Hardness, 3 for Elongation
};
```

```
void addTest(struct SteelTest **tests, int *testCount);
void displayTests(struct SteelTest *tests, int testCount);
```

```
int main()
{
    struct SteelTest *tests = NULL;
    int testCount = 0, option;
    do
    {
        printf("\nSteel Quality Monitoring System\n");
        printf("1. Add Test Record\n");
        printf("2. Display Test Records\n");
        printf("3. Exit\n");
        printf("Enter your option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addTest(&tests, &testCount);
                    break;
            case 2: displayTests(tests, testCount);
                    break;
```

```

        case 3: for (int i = 0; i < testCount; i++)
            free((int *)tests[i].testID);
        free(tests);
        printf("Exiting the system\n");
        break;
        default: printf("Invalid option\n");
    }
} while (option != 3);
return 0;
}

void addTest(struct SteelTest **tests, int *testCount)
{
    int tempTestID;
    struct SteelTest newTest;
    printf("Enter Test ID: ");
    scanf("%d", &tempTestID);
    int *testIDPtr = (int *)malloc(sizeof(int));
    *testIDPtr = tempTestID;
    newTest.testID = testIDPtr;
    printf("Enter Test Type: ");
    scanf(" %[^\\n]", newTest.testType);
    printf("Enter Metric Type (1 for Tensile Strength, 2 for Hardness, 3 for
                                                    Elongation): ");
    scanf("%d", &newTest.metricType);
    switch (newTest.metricType)
    {

```

```

        case 1: printf("Enter Tensile Strength: ");
                scanf("%f", &newTest.result.tensileStrength);
                break;
        case 2: printf("Enter Hardness: ");
                scanf("%f", &newTest.result.hardness);
                break;
        case 3: printf("Enter Elongation: ");
                scanf("%f", &newTest.result.elongation);
                break;
        default: printf("Invalid metric type\n");
                return;
    }

    *tests = (struct SteelTest *)realloc(*tests, (*testCount + 1) *
                                          sizeof(struct SteelTest));

    (*tests)[*testCount] = newTest;
    (*testCount)++;
    printf("Steel quality test record added successfully\n");
}

void displayTests(struct SteelTest *tests, int testCount)
{
    if (testCount == 0)
    {
        printf("No test records available\n");
        return;
    }

    printf("\nSteel Quality Test Records\n");

```

```

for (int i = 0; i < testCount; i++)
{
    printf("Test ID: %d\n", *(tests[i].testID));
    printf("Test Type: %s\n", tests[i].testType);
    switch (tests[i].metricType)
    {
        case 1: printf("Metric: Tensile Strength\n");
                printf("Result: %.2f MPa\n", tests[i].result.tensileStrength);
                break;
        case 2: printf("Metric: Hardness\n");
                printf("Result: %.2f HBW\n", tests[i].result.hardness);
                break;
        case 3: printf("Metric: Elongation\n");
                printf("Result: %.2f%%\n", tests[i].result.elongation);
                break;
        default: printf("Invalid metric type\n");
    }
}
}

```

4. Metal Fatigue Analysis

Description:

Develop a program to analyze metal fatigue using arrays for stress cycle data, structures for material details, and strings for material names.

Specifications:

- Structure: Contains material ID, name, and endurance limit.
- Array: Stress cycle data.

- Strings: Material names.
- const Pointers: Protect material details.
- Double Pointers: Allocate dynamic material test data.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Material
{
    const int *materialID;
    char materialName[50];
    float enduranceLimit;
    int stressCycleCount;
    float *stressCycles;
};
```

```
void addMaterial(struct Material **materials, int *materialCount);
void displayMaterials(struct Material *materials, int materialCount);
```

```
int main()
{
    struct Material *materials = NULL;
    int materialCount = 0, option;
    do
    {
        printf("\nMetal Fatigue Analysis System\n");
```

```

printf("1. Add Material Data\n");
printf("2. Display Material Data\n");
printf("3. Exit\n");
printf("Enter your option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addMaterial(&materials, &materialCount);
            break;
    case 2: displayMaterials(materials, materialCount);
            break;
    case 3: for (int i = 0; i < materialCount; i++)
            {
                free((int *)materials[i].materialID);
                free(materials[i].stressCycles);
            }
            free(materials);
            printf("Exiting the system\n");
            break;
    default:printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

void addMaterial(struct Material **materials, int *materialCount)
{

```

```

int tempMaterialID, cycleCount;
struct Material newMaterial;
printf("Enter Material ID: ");
scanf("%d", &tempMaterialID);
int *materialIDPtr = (int *)malloc(sizeof(int));
*materialIDPtr = tempMaterialID;
newMaterial.materialID = materialIDPtr;
printf("Enter Material Name: ");
scanf(" %[^\\n]", newMaterial.materialName);
printf("Enter Endurance Limit: ");
scanf("%f", &newMaterial.enduranceLimit);
printf("Enter the number of stress cycles: ");
scanf("%d", &cycleCount);
newMaterial.stressCycleCount = cycleCount;
newMaterial.stressCycles = (float *)malloc(cycleCount * sizeof(float));
printf("Enter stress cycle data:\\n");
for (int i = 0; i < cycleCount; i++)
{
    printf("Cycle %d: ", i + 1);
    scanf("%f", &newMaterial.stressCycles[i]);
}
*materials = (struct Material *)realloc(*materials, (*materialCount + 1) *
                                         sizeof(struct Material));
(*materials)[*materialCount] = newMaterial;
(*materialCount)++;
printf("Material data added successfully\\n");
}

```

```

void displayMaterials(struct Material *materials, int materialCount)
{
    if (materialCount == 0)
    {
        printf("No material data available\n");
        return;
    }
    printf("\nMaterial Fatigue Data\n");
    for (int i = 0; i < materialCount; i++)
    {
        printf("Material ID: %d\n", *(materials[i].materialID));
        printf("Material Name: %s\n", materials[i].materialName);
        printf("Endurance Limit: %.2f MPa\n", materials[i].enduranceLimit);
        printf("Stress Cycles: ");
        for (int j = 0; j < materials[i].stressCycleCount; j++)
            printf("%.2f ", materials[i].stressCycles[j]);
    }
}

```

5. Foundry Management System

Description:

Create a system for managing foundry operations using arrays for equipment data, structures for casting details, and unions for variable mold properties.

Specifications:

- Structure: Stores casting ID, weight, and material.
- Union: Represents mold properties (dimensions or thermal conductivity).
- Array: Equipment data.

- const Pointers: Protect equipment details.
- Double Pointers: Dynamic allocation of casting records.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Casting
```

```
{
```

```
    const int *castingID;
```

```
    float weight;
```

```
    char material[50];
```

```
    union
```

```
    {
```

```
        struct
```

```
        {
```

```
            float length;
```

```
            float width;
```

```
            float height;
```

```
        } dimensions;
```

```
        float thermalConductivity;
```

```
    } moldProperties;
```

```
    int propertyType; // 1 for dimensions, 2 for thermal conductivity
```

```
};
```

```
void addCasting(struct Casting **castings, int *castingCount);
```

```
void displayCastings(struct Casting *castings, int castingCount);
```

```

int main()
{
    struct Casting *castings = NULL;
    int castingCount = 0, option;
    do {
        printf("\nFoundry Management System\n");
        printf("1. Add Casting Record\n");
        printf("2. Display Casting Records\n");
        printf("3. Exit\n");
        printf("Enter your option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addCasting(&castings, &castingCount);
                    break;
            case 2: displayCastings(castings, castingCount);
                    break;
            case 3: for (int i = 0; i < castingCount; i++)
                    free((int *)castings[i].castingID);
                    free(castings);
                    printf("Exiting the system\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

```

```

void addCasting(struct Casting **castings, int *castingCount)
{
    int tempCastingID;
    struct Casting newCasting;
    printf("Enter Casting ID: ");
    scanf("%d", &tempCastingID);
    int *castingIDPtr = (int *)malloc(sizeof(int));
    *castingIDPtr = tempCastingID;
    newCasting.castingID = castingIDPtr;
    printf("Enter Weight (in kilograms): ");
    scanf("%f", &newCasting.weight);
    printf("Enter Material: ");
    scanf(" %[^\n]", newCasting.material);
    printf("Enter Property Type (1 for Dimensions, 2 for Thermal Conductivity):
                                                ");
    scanf("%d", &newCasting.propertyType);
    switch (newCasting.propertyType)
    {
        case 1: printf("Enter Length: ");
                scanf("%f", &newCasting.moldProperties.dimensions.length);
                printf("Enter Width: ");
                scanf("%f", &newCasting.moldProperties.dimensions.width);
                printf("Enter Height: ");
                scanf("%f", &newCasting.moldProperties.dimensions.height);
                break;
        case 2: printf("Enter Thermal Conductivity: ");
                scanf("%f", &newCasting.moldProperties.thermalConductivity);
    }
}

```

```

        break;
    default: printf("Invalid property type\n");
        return;
    }
    *castings = (struct Casting *)realloc(*castings, (*castingCount + 1) *
                                           sizeof(struct Casting));
    (*castings)[*castingCount] = newCasting;
    (*castingCount)++;
    printf("Casting record added successfully\n");
}

```

```

void displayCastings(struct Casting *castings, int castingCount)
{
    if (castingCount == 0)
    {
        printf("No casting records available\n");
        return;
    }
    printf("\nCasting Records\n");
    for (int i = 0; i < castingCount; i++)
    {
        printf("Casting ID: %d\n", *(castings[i].castingID));
        printf("Weight: %.2f kg\n", castings[i].weight);
        printf("Material: %s\n", castings[i].material);
        switch (castings[i].propertyType)
        {
            case 1: printf("Mold Properties: Dimensions\n");

```

```

        printf("Length: %.2f m\n",
               castings[i].moldProperties.dimensions.length);
        printf("Width: %.2f m\n",
               castings[i].moldProperties.dimensions.width);
        printf("Height: %.2f m\n",
               castings[i].moldProperties.dimensions.height);
        break;
    case 2: printf("Mold Properties: Thermal Conductivity\n");
        printf("Thermal Conductivity: %.2f W/(m·K)\n",
               castings[i].moldProperties.thermalConductivity);
        break;
    default: printf("Invalid property type\n");
}
}
}

```

6. Metal Purity Analysis

Description:

Develop a system for metal purity analysis using structures for sample data, arrays for impurity percentages, and unions for variable impurity types.

Specifications:

- Structure: Contains sample ID, type, and purity.
- Union: Represents impurity type (trace elements or oxides).
- Array: Impurity percentages.
- const Pointers: Protect purity data.
- Double Pointers: Manage dynamic impurity records.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Sample
{
    const int *sampleID;
    char sampleType[50];
    const float *purity;
    union
    {
        struct
        {
            int elementCount;
            char elements[10][30];
        } traceElements;
        struct
        {
            int oxideCount;
            char oxides[10][30];
        } oxides;
    } impurityType;
    int impurityCategory;
    float *impurityPercentages;
    int impurityCount;
};
```

```
void addSample(struct Sample **samples, int *sampleCount);  
void displaySamples(struct Sample *samples, int sampleCount);
```

```
int main()  
{  
    struct Sample *samples = NULL;  
    int sampleCount = 0, option;  
    do  
    {  
        printf("\nMetal Purity Analysis System\n");  
        printf("1. Add Sample Data\n");  
        printf("2. Display Sample Data\n");  
        printf("3. Exit\n");  
        printf("Enter your option: ");  
        scanf("%d", &option);  
        switch (option)  
        {  
            case 1: addSample(&samples, &sampleCount);  
                    break;  
            case 2: displaySamples(samples, sampleCount);  
                    break;  
            case 3: for (int i = 0; i < sampleCount; i++)  
                    {  
                        free((int *)samples[i].sampleID);  
                        free((float *)samples[i].purity);  
                        free(samples[i].impurityPercentages);  
                    }  
        }  
    }  
}
```

```

        free(samples);
        printf("Exiting\n");
        break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addSample(struct Sample **samples, int *sampleCount)
{
    int tempSampleID, impurityCategory, impurityCount;
    float tempPurity;
    struct Sample newSample;
    printf("Enter Sample ID: ");
    scanf("%d", &tempSampleID);
    int *sampleIDPtr = (int *)malloc(sizeof(int));
    *sampleIDPtr = tempSampleID;
    newSample.sampleID = sampleIDPtr;
    printf("Enter Sample Type: ");
    scanf(" %[^\\n]", newSample.sampleType);
    printf("Enter Purity Percentage: ");
    scanf("%f", &tempPurity);
    float *purityPtr = (float *)malloc(sizeof(float));
    *purityPtr = tempPurity;
    newSample.purity = purityPtr;
    printf("Enter Impurity Category (1 for Trace Elements, 2 for Oxides): ");
}

```

```

scanf("%d", &impurityCategory);
newSample.impurityCategory = impurityCategory;
printf("Enter Number of Impurities: ");
scanf("%d", &impurityCount);
newSample.impurityCount = impurityCount;
newSample.impurityPercentages = (float *)malloc(impurityCount *
                                                sizeof(float));

switch (impurityCategory)
{
    case 1: newSample.impurityType.traceElements.elementCount =
                                                    impurityCount;

        printf("Enter Names of Trace Elements:\n");
        for (int i = 0; i < impurityCount; i++)
        {
            printf("Element %d: ", i + 1);
            scanf(" %[^\n]",
                newSample.impurityType.traceElements.elements[i]);
            printf("Enter Percentage for %s: ",
                newSample.impurityType.traceElements.elements[i]);
            scanf("%f", &newSample.impurityPercentages[i]);
        }
        break;

    case 2: newSample.impurityType.oxides.oxideCount = impurityCount;

        printf("Enter Names of Oxides:\n");
        for (int i = 0; i < impurityCount; i++)
        {
            printf("Oxide %d: ", i + 1);

```

```

        scanf(" %[^\\n]", newSample.impurityType.oxides.oxides[i]);
        printf("Enter Percentage for %s: ",
               newSample.impurityType.oxides.oxides[i]);
        scanf("%f", &newSample.impurityPercentages[i]);
    }
    break;
default:printf("Invalid impurity category\\n");
    free(sampleIDPtr);
    free(purityPtr);
    free(newSample.impurityPercentages);
    return;
}
*samples = (struct Sample *)realloc(*samples, (*sampleCount + 1) *
                                     sizeof(struct Sample));

(*samples)[*sampleCount] = newSample;
(*sampleCount)++;
printf("Sample data added successfully\\n");
}

void displaySamples(struct Sample *samples, int sampleCount)
{
    if (sampleCount == 0)
    {
        printf("No sample data available\\n");
        return;
    }
    printf("\\nSample Data:\\n");

```

```

for (int i = 0; i < sampleCount; i++)
{
    printf("Sample ID: %d\n", *(samples[i].sampleID));
    printf("Sample Type: %s\n", samples[i].sampleType);
    printf("Purity: %.2f%%\n", *(samples[i].purity));
    printf("Impurities:\n");
    switch (samples[i].impurityCategory)
    {
        case 1: printf("Trace Elements:\n");
                for (int j = 0; j < samples[i].
                    impurityType.traceElements.elementCount; j++)
                    printf("  %s: %.2f%%\n",
                        samples[i].impurityType.traceElements.elements[j],
                        samples[i].impurityPercentages[j]);
                break;
        case 2: printf("Oxides:\n");
                for (int j = 0; j < samples[i].impurityType.oxides.oxideCount; j++)
                    printf("  %s: %.2f%%\n",
                        samples[i].impurityType.oxides.oxides[j],
                        samples[i].impurityPercentages[j]);
                break;
        default: printf("Invalid impurity category\n");
    }
}
}

```

7. Corrosion Testing System

Description:

Create a program to track corrosion tests using structures for test details, arrays for test results, and strings for test conditions.

Specifications:

- Structure: Holds test ID, duration, and environment.
- Array: Test results.
- Strings: Test conditions.
- const Pointers: Protect test configurations.
- Double Pointers: Dynamic allocation of test records.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct CorrosionTest
```

```
{
```

```
    const int *testID;
```

```
    const int *duration;
```

```
    char environment[50];
```

```
    char testCondition[100];
```

```
    float *testResults;
```

```
    int resultCount;
```

```
};
```

```
void addTest(struct CorrosionTest **tests, int *testCount);
```

```
void displayTests(struct CorrosionTest *tests, int testCount);
```

```

int main()
{
    struct CorrosionTest *tests = NULL;
    int testCount = 0, option;
    do
    {
        printf("\nCorrosion Testing System\n");
        printf("1. Add Corrosion Test\n");
        printf("2. Display Corrosion Tests\n");
        printf("3. Exit\n");
        printf("Enter your option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addTest(&tests, &testCount);
                    break;
            case 2: displayTests(tests, testCount);
                    break;
            case 3: for (int i = 0; i < testCount; i++)
                    {
                        free((int *)tests[i].testID);
                        free((int *)tests[i].duration);
                        free(tests[i].testResults);
                    }
                    free(tests);
                    printf("Exiting the system\n");
                    break;
        }
    }
}

```

```

        default:printf("Invalid option\n");
    }
} while (option != 3);
return 0;
}

```

```

void addTest(struct CorrosionTest **tests, int *testCount)
{
    int tempTestID, tempDuration, resultCount;
    struct CorrosionTest newTest;
    printf("Enter Test ID: ");
    scanf("%d", &tempTestID);
    int *testIDPtr = (int *)malloc(sizeof(int));
    *testIDPtr = tempTestID;
    newTest.testID = testIDPtr;
    printf("Enter Test Duration (in hours): ");
    scanf("%d", &tempDuration);
    int *durationPtr = (int *)malloc(sizeof(int));
    *durationPtr = tempDuration;
    newTest.duration = durationPtr;
    printf("Enter Test Environment: ");
    scanf(" %[^\\n]", newTest.environment);
    printf("Enter Test Conditions: ");
    scanf(" %[^\\n]", newTest.testCondition);
    printf("Enter Number of Test Results: ");
    scanf("%d", &resultCount);
    newTest.resultCount = resultCount;
}

```

```

newTest.testResults = (float *)malloc(resultCount * sizeof(float));
printf("Enter Test Results:\n");
for (int i = 0; i < resultCount; i++)
{
    printf("Result %d: ", i + 1);
    scanf("%f", &newTest.testResults[i]);
}
*tests = (struct CorrosionTest *)realloc(*tests, (*testCount + 1) * sizeof(struct
                                         CorrosionTest));

(*tests)[*testCount] = newTest;
(*testCount)++;
printf("Corrosion test added successfully\n");
}

```

```

void displayTests(struct CorrosionTest *tests, int testCount)
{
    if (testCount == 0)
    {
        printf("No corrosion test data available\n");
        return;
    }
    printf("\nCorrosion Test Data:\n");
    for (int i = 0; i < testCount; i++)
    {
        printf("Test ID: %d\n", *(tests[i].testID));
        printf("Duration: %d hours\n", *(tests[i].duration));
        printf("Environment: %s\n", tests[i].environment);
    }
}

```

```

    printf("Test Conditions: %s\n", tests[i].testCondition);
    printf("Test Results:\n");
    for (int j = 0; j < tests[i].resultCount; j++)
        printf("Result %d: %.2f\n", j + 1, tests[i].testResults[j]);
    }
}

```

8. Welding Parameter Optimization

Description:

Develop a program to optimize welding parameters using structures for parameter sets, arrays for test outcomes, and unions for variable welding types.

Specifications:

- Structure: Stores parameter ID, voltage, current, and speed.
- Union: Represents welding types (MIG, TIG, or Arc).
- Array: Test outcomes.
- const Pointers: Protect parameter configurations.
- Double Pointers: Manage dynamic parameter sets.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
union WeldingType
```

```
{
```

```
    char MIG[20]; // Metal Inert Gas Welding
```

```
    char TIG[20]; // Tungsten Inert Gas Welding
```

```
    char Arc[20]; // Arc Welding
```

```
};
```

```
struct WeldingParameters
```

```
{
```

```
    const int *parameterID;
```

```
    const float *voltage;
```

```
    const float *current;
```

```
    const float *speed;
```

```
    union WeldingType type;
```

```
    int typeCode;          // Type code (1 for MIG, 2 for TIG, 3 for Arc)
```

```
    float *testOutcomes;
```

```
    int outcomeCount;
```

```
};
```

```
void addParameterSet(struct WeldingParameters **parameters, int *count);
```

```
void displayParameterSets(struct WeldingParameters *parameters, int count);
```

```
int main()
```

```
{
```

```
    struct WeldingParameters *parameters = NULL;
```

```
    int parameterCount = 0, option;
```

```
    do
```

```
    {
```

```
        printf("\nWelding Parameter Optimization\n");
```

```
        printf("1. Add Welding Parameter Set\n");
```

```
        printf("2. Display Welding Parameter Sets\n");
```

```
        printf("3. Exit\n");
```

```

printf("Enter the option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addParameterSet(&parameters, &parameterCount);
            break;
    case 2: displayParameterSets(parameters, parameterCount);
            break;
    case 3: for (int i = 0; i < parameterCount; i++)
            {
                free((int *)parameters[i].parameterID);
                free((float *)parameters[i].voltage);
                free((float *)parameters[i].current);
                free((float *)parameters[i].speed);
                free(parameters[i].testOutcomes);
            }
            free(parameters);
            printf("Exiting\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

void addParameterSet(struct WeldingParameters **parameters, int *count)
{

```

```
int tempID, typeCode, outcomeCount;
float tempVoltage, tempCurrent, tempSpeed;
struct WeldingParameters newSet;
printf("Enter Parameter ID: ");
scanf("%d", &tempID);
int *idPtr = (int *)malloc(sizeof(int));
*idPtr = tempID;
newSet.parameterID = idPtr;
printf("Enter Voltage: ");
scanf("%f", &tempVoltage);
float *voltagePtr = (float *)malloc(sizeof(float));
*voltagePtr = tempVoltage;
newSet.voltage = voltagePtr;
printf("Enter Current: ");
scanf("%f", &tempCurrent);
float *currentPtr = (float *)malloc(sizeof(float));
*currentPtr = tempCurrent;
newSet.current = currentPtr;
printf("Enter Speed: ");
scanf("%f", &tempSpeed);
float *speedPtr = (float *)malloc(sizeof(float));
*speedPtr = tempSpeed;
newSet.speed = speedPtr;
printf("Enter Welding Type (1 for MIG, 2 for TIG, 3 for Arc): ");
scanf("%d", &typeCode);
newSet.typeCode = typeCode;
switch (typeCode)
```

```

{
    case 1: printf("Enter MIG Welding Details: ");
        scanf(" %[^\\n]", newSet.type.MIG);
        break;
    case 2: printf("Enter TIG Welding Details: ");
        scanf(" %[^\\n]", newSet.type.TIG);
        break;
    case 3: printf("Enter Arc Welding Details: ");
        scanf(" %[^\\n]", newSet.type.Arc);
        break;
    default: printf("Invalid Welding Type\\n");
        return;
}

printf("Enter Number of Test Outcomes: ");
scanf("%d", &outcomeCount);
newSet.outcomeCount = outcomeCount;
newSet.testOutcomes = (float *)malloc(outcomeCount * sizeof(float));
printf("Enter Test Outcomes:\\n");
for (int i = 0; i < outcomeCount; i++)
{
    printf("Outcome %d: ", i + 1);
    scanf("%f", &newSet.testOutcomes[i]);
}

*parameters = (struct WeldingParameters *)realloc(*parameters, (*count + 1)
                                                * sizeof(struct WeldingParameters));

(*parameters)[*count] = newSet;
(*count)++;

```

```

    printf("Welding parameter set added successfully\n");
}

void displayParameterSets(struct WeldingParameters *parameters, int count)
{
    if (count == 0)
    {
        printf("No welding parameter sets available\n");
        return;
    }
    printf("\nWelding Parameter Sets\n");
    for (int i = 0; i < count; i++)
    {
        printf("Parameter ID: %d\n", *(parameters[i].parameterID));
        printf("Voltage: %.2f V\n", *(parameters[i].voltage));
        printf("Current: %.2f A\n", *(parameters[i].current));
        printf("Speed: %.2f mm/s\n", *(parameters[i].speed));
        switch (parameters[i].typeCode)
        {
            case 1: printf("Welding Type: MIG - %s\n", parameters[i].type.MIG);
                    break;
            case 2: printf("Welding Type: TIG - %s\n", parameters[i].type.TIG);
                    break;
            case 3: printf("Welding Type: Arc - %s\n", parameters[i].type.Arc);
                    break;
            default: printf("Welding Type: Unknown\n");
        }
    }
}

```

```

    printf("Test Outcomes:\n");
    for (int j = 0; j < parameters[i].outcomeCount; j++)
        printf(" Outcome %d: %.2f\n", j + 1, parameters[i].testOutcomes[j]);
    }
}

```

9. Metal Surface Finish Analysis

Description:

Design a program to analyze surface finishes using arrays for measurement data, structures for test configurations, and strings for surface types.

Specifications:

- Structure: Holds configuration ID, material, and measurement units.
- Array: Surface finish measurements.
- Strings: Surface types.
- const Pointers: Protect configuration details.
- Double Pointers: Allocate and manage measurement data.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct SurfaceFinishConfig
```

```
{
```

```
    const int *configID;
```

```
    const char *material;
```

```
    const char *units;
```

```
    float *measurements;
```

```

    int measurementCount;
    char surfaceType[30];
};

void addSurfaceFinishConfig(struct SurfaceFinishConfig **configs, int *count);
void displaySurfaceFinishConfigs(struct SurfaceFinishConfig *configs,
                                int count);

int main()
{
    struct SurfaceFinishConfig *configs = NULL;
    int configCount = 0, option;
    do
    {
        printf("\nMetal Surface Finish Analysis\n");
        printf("1. Add Surface Finish Configuration\n");
        printf("2. Display Surface Finish Configurations\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addSurfaceFinishConfig(&configs, &configCount);
                    break;
            case 2: displaySurfaceFinishConfigs(configs, configCount);
                    break;
            case 3: for (int i = 0; i < configCount; i++)

```

```

        {
            free((int *)configs[i].configID);
            free((char *)configs[i].material);
            free((char *)configs[i].units);
            free(configs[i].measurements);
        }
        free(configs);
        printf("Exiting\n");
        break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addSurfaceFinishConfig(struct SurfaceFinishConfig **configs, int *count)
{
    int tempID, measurementCount;
    char tempMaterial[50], tempUnits[20], tempSurfaceType[30];
    struct SurfaceFinishConfig newConfig;
    printf("Enter Configuration ID: ");
    scanf("%d", &tempID);
    int *idPtr = (int *)malloc(sizeof(int));
    *idPtr = tempID;
    newConfig.configID = idPtr;
    printf("Enter Material: ");
    scanf(" %[^\n]", tempMaterial);
}

```

```

char *materialPtr = (char *)malloc((strlen(tempMaterial) + 1) * sizeof(char));
strcpy(materialPtr, tempMaterial);
newConfig.material = materialPtr;
printf("Enter Measurement Units: ");
scanf(" %[^\\n]", tempUnits);
char *unitsPtr = (char *)malloc((strlen(tempUnits) + 1) * sizeof(char));
strcpy(unitsPtr, tempUnits);
newConfig.units = unitsPtr;
printf("Enter Surface Type: ");
scanf(" %[^\\n]", newConfig.surfaceType);
printf("Enter Number of Measurements: ");
scanf("%d", &measurementCount);
newConfig.measurementCount = measurementCount;
newConfig.measurements = (float *)malloc(measurementCount *
                                          sizeof(float));

printf("Enter Measurements:\\n");
for (int i = 0; i < measurementCount; i++)
{
    printf("Measurement %d: ", i + 1);
    scanf("%f", &newConfig.measurements[i]);
}
*configs = (struct SurfaceFinishConfig *)realloc(*configs, (*count + 1) *
                                                  sizeof(struct SurfaceFinishConfig));
(*configs)[*count] = newConfig;
(*count)++;
printf("Surface finish configuration added successfully\\n");
}

```

```

void displaySurfaceFinishConfigs(struct SurfaceFinishConfig *configs,
                                int count)
{
    if (count == 0)
    {
        printf("No surface finish configurations available\n");
        return;
    }
    printf("\nSurface Finish Configurations\n");
    for (int i = 0; i < count; i++)
    {
        printf("Configuration ID: %d\n", *(configs[i].configID));
        printf("Material: %s\n", configs[i].material);
        printf("Measurement Units: %s\n", configs[i].units);
        printf("Surface Type: %s\n", configs[i].surfaceType);
        printf("Measurements:\n");
        for (int j = 0; j < configs[i].measurementCount; j++)
            printf("Measurement %d: %.2f\n", j + 1, configs[i].measurements[j]);
    }
}

```

10. Smelting Process Tracker

Description:

Create a system to track smelting processes using structures for process metadata, arrays for heat data, and unions for variable ore properties.

Specifications:

- Structure: Holds process ID, ore type, and temperature.

- Union: Represents variable ore properties.
- Array: Heat data.
- const Pointers: Protect process metadata.
- Double Pointers: Allocate dynamic process records.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
union OreProperties
```

```
{
```

```
    float density;
```

```
    float purity;
```

```
    char grade[20];
```

```
};
```

```
struct SmeltingProcess
```

```
{
```

```
    const int *processID;
```

```
    const char *oreType;
```

```
    float temperature;
```

```
    union OreProperties properties;
```

```
    float *heatData;
```

```
    int heatDataCount;
```

```
};
```

```
void addSmeltingProcess(struct SmeltingProcess **processes, int *count);
```

```
void displaySmeltingProcesses(struct SmeltingProcess *processes, int count);
```

```
int main()
```

```
{
```

```
    struct SmeltingProcess *processes = NULL;
```

```
    int processCount = 0, option;
```

```
    do
```

```
    {
```

```
        printf("\nSmelting Process Tracker\n");
```

```
        printf("1. Add Smelting Process\n");
```

```
        printf("2. Display Smelting Processes\n");
```

```
        printf("3. Exit\n");
```

```
        printf("Enter the option: ");
```

```
        scanf("%d", &option);
```

```
        switch (option)
```

```
        {
```

```
            case 1: addSmeltingProcess(&processes, &processCount);
```

```
                break;
```

```
            case 2: displaySmeltingProcesses(processes, processCount);
```

```
                break;
```

```
            case 3: for (int i = 0; i < processCount; i++)
```

```
                {
```

```
                    free((int *)processes[i].processID);
```

```
                    free((char *)processes[i].oreType);
```

```
                    free(processes[i].heatData);
```

```
                }
```

```
            free(processes);
```

```

        printf("Exiting the tracker\n");
        break;
    default:printf("Invalid option\n");
    }
} while (option != 3);
return 0;
}

```

```

void addSmeltingProcess(struct SmeltingProcess **processes, int *count)
{
    int tempID, heatCount;
    char tempOreType[50], tempGrade[20];
    struct SmeltingProcess newProcess;
    printf("Enter Process ID: ");
    scanf("%d", &tempID);
    int *idPtr = (int *)malloc(sizeof(int));
    *idPtr = tempID;
    newProcess.processID = idPtr;
    printf("Enter Ore Type: ");
    scanf(" %[^n]", tempOreType);
    char *oreTypePtr = (char *)malloc((strlen(tempOreType) + 1) * sizeof(char));
    strcpy(oreTypePtr, tempOreType);
    newProcess.oreType = oreTypePtr;
    printf("Enter Smelting Temperature: ");
    scanf("%f", &newProcess.temperature);
    printf("Enter Ore Property Type (1 for Density, 2 for Purity, 3 for Grade): ");
    int propertyType;

```

[illegible]

```

(*processes)[*count] = newProcess;
(*count)++;
printf("Smelting process added successfully\n");
}

void displaySmeltingProcesses(struct SmeltingProcess *processes, int count)
{
    if (count == 0)
    {
        printf("No smelting processes available\n");
        return;
    }
    printf("\nSmelting Processes\n");
    for (int i = 0; i < count; i++)
    {
        printf("Process ID: %d\n", *(processes[i].processID));
        printf("Ore Type: %s\n", processes[i].oreType);
        printf("Smelting Temperature: %.2f\n", processes[i].temperature);
        printf("Ore Properties: ");
        if (strlen(processes[i].properties.grade) > 0)
            printf("Grade - %s\n", processes[i].properties.grade);
        else if (processes[i].properties.purity > 0)
            printf("Purity - %.2f\n", processes[i].properties.purity);
        else
            printf("Density - %.2f\n", processes[i].properties.density);
        printf("Heat Data\n");
        for (int j = 0; j < processes[i].heatDataCount; j++)

```

```
        printf("Heat Data Point %d: %.2f\n", j + 1, processes[i].heatData[j]);
    }
}
```

11. Electroplating System Simulation

Description:

Simulate an electroplating system using structures for metal ions, arrays for plating parameters, and strings for electrolyte names.

Specifications:

- Structure: Stores ion type, charge, and concentration.
- Array: Plating parameters.
- Strings: Electrolyte names.
- const Pointers: Protect ion data.
- Double Pointers: Manage dynamic plating configurations.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct IonData
{
    const char *ionType;
    const char *charge;
    float concentration;
};
```

```
struct ElectroplatingConfig
```

```
{  
    const struct IonData *ionData;  
    const char *electrolyteName;  
    float *platingParameters;  
    int parameterCount;  
};
```

```
void addElectroplatingConfig(struct ElectroplatingConfig **configs, int *count);  
void displayElectroplatingConfigs(struct ElectroplatingConfig *configs,  
                                   int count);
```

```
int main()  
{  
    struct ElectroplatingConfig *configs = NULL;  
    int configCount = 0, option;  
    do  
    {  
        printf("\nElectroplating System Simulation\n");  
        printf("1. Add Electroplating Configuration\n");  
        printf("2. Display Electroplating Configurations\n");  
        printf("3. Exit\n");  
        printf("Enter the option: ");  
        scanf("%d", &option);  
        switch (option)  
        {  
            case 1: addElectroplatingConfig(&configs, &configCount);  
                    break;
```

```

        case 2: displayElectroplatingConfigs(configs, configCount);
            break;
        case 3: for (int i = 0; i < configCount; i++)
            {
                free((char *)configs[i].ionData->ionType);
                free((char *)configs[i].ionData->charge);
                free(configs[i].platingParameters);
                free((char *)configs[i].electrolyteName);
                free((struct IonData *)configs[i].ionData);
            }
            free(configs);
            printf("Exiting\n");
            break;
        default: printf("Invalid option\n");
    }
} while (option != 3);
return 0;
}

void addElectroplatingConfig(struct ElectroplatingConfig **configs, int *count)
{
    struct IonData *ionData = (struct IonData *)malloc(sizeof(struct IonData));
    struct ElectroplatingConfig newConfig;
    int parameterCount;
    char ionType[50], charge[10];
    printf("Enter Ion Type (e.g., Cu2+): ");
    scanf("%s", ionType);

```

```

ionData->ionType = (char *)malloc((strlen(ionType) + 1) * sizeof(char));
strcpy((char *)ionData->ionType, ionType);
printf("Enter Ion Charge (e.g., +2): ");
scanf("%s", charge);
ionData->charge = (char *)malloc((strlen(charge) + 1) * sizeof(char));
strcpy((char *)ionData->charge, charge);
printf("Enter Ion Concentration (e.g., 0.1 M): ");
scanf("%f", &ionData->concentration);
newConfig.ionData = ionData;
char electrolyteName[50];
printf("Enter Electrolyte Name (e.g., Copper Sulfate): ");
scanf(" %[^\\n]", electrolyteName);
newConfig.electrolyteName = (char *)malloc((strlen(electrolyteName) + 1) *
                                            sizeof(char));
strcpy((char *)newConfig.electrolyteName, electrolyteName);
printf("Enter Number of Plating Parameters (e.g., Current, Time, etc.): ");
scanf("%d", &parameterCount);
newConfig.parameterCount = parameterCount;
newConfig.platingParameters = (float *)malloc(parameterCount *
                                                sizeof(float));

printf("Enter Plating Parameters:\\n");
for (int i = 0; i < parameterCount; i++)
{
    printf("Parameter %d: ", i + 1);
    scanf("%f", &newConfig.platingParameters[i]);
}

*configs = (struct ElectroplatingConfig *)realloc(*configs, (*count + 1) *

```

```

        sizeof(struct ElectroplatingConfig));

(*configs)[*count] = newConfig;
(*count)++;
printf("Electroplating configuration added successfully\n");
}

void displayElectroplatingConfigs(struct ElectroplatingConfig *configs,
                                int count)
{
    if (count == 0)
    {
        printf("No electroplating configurations available\n");
        return;
    }
    printf("\nElectroplating Configurations\n");
    for (int i = 0; i < count; i++)
    {
        printf("Ion Type: %s\n", configs[i].ionData->ionType);
        printf("Ion Charge: %s\n", configs[i].ionData->charge);
        printf("Ion Concentration: %.2f M\n", configs[i].ionData->concentration);
        printf("Electrolyte Name: %s\n", configs[i].electrolyteName);
        printf("Plating Parameters:\n");
        for (int j = 0; j < configs[i].parameterCount; j++)
            printf("Parameter %d: %.2f\n", j + 1, configs[i].platingParameters[j]);
    }
}

```

12. Casting Defect Analysis

Description:

Design a system to analyze casting defects using arrays for defect data, structures for casting details, and unions for variable defect types.

Specifications:

- Structure: Holds casting ID, material, and dimensions.
- Union: Represents defect types (shrinkage or porosity).
- Array: Defect data.
- const Pointers: Protect casting data.
- Double Pointers: Dynamic defect record management.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
union DefectData
```

```
{
```

```
    struct
```

```
    {
```

```
        float shrinkageRatio;
```

```
    } shrinkage;
```

```
    struct
```

```
    {
```

```
        float porosityPercentage;
```

```
    } porosity;
```

```
};
```

```
struct CastingDetails
```

```

{
    const int *castingID;
    const char *material;
    float dimensions[3];
};

struct DefectRecord
{
    const struct CastingDetails *castingDetails;
    union DefectData defect;
    char defectType[20]; // shrinkage or porosity
};

void addDefectRecord(struct DefectRecord **defects, int *defectCount);
void displayDefectRecords(struct DefectRecord *defects, int defectCount);

int main()
{
    struct DefectRecord *defects = NULL;
    int defectCount = 0, option;
    do
    {
        printf("\nCasting Defect Analysis\n");
        printf("1. Add Defect Record\n");
        printf("2. Display Defect Records\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
    }

```

```

scanf("%d", &option);
switch (option)
{
    case 1: addDefectRecord(&defects, &defectCount);
            break;
    case 2: displayDefectRecords(defects, defectCount);
            break;
    case 3: for (int i = 0; i < defectCount; i++)
            {
                free((int *)defects[i].castingDetails->castingID);
                free((char *)defects[i].castingDetails->material);
                free((struct CastingDetails *)defects[i].castingDetails);
            }
            free(defects);
            printf("Exiting\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addDefectRecord(struct DefectRecord **defects, int *defectCount)
{
    struct CastingDetails *casting = (struct CastingDetails *)malloc(sizeof(struct
                                                                    CastingDetails));

    struct DefectRecord newDefect;

```

```

int castingID;
char material[50], defectType[20];
float length, width, height;
float defectValue;
printf("Enter Casting ID: ");
scanf("%d", &castingID);
int *castingIDPtr = (int *)malloc(sizeof(int));
*castingIDPtr = castingID;
casting->castingID = castingIDPtr;
printf("Enter Material: ");
scanf(" %[^\\n]", material);
casting->material = (char *)malloc((strlen(material) + 1) * sizeof(char));
strcpy((char *)casting->material, material);
printf("Enter Dimensions (Length, Width, Height): ");
scanf("%f %f %f", &length, &width, &height);
casting->dimensions[0] = length;
casting->dimensions[1] = width;
casting->dimensions[2] = height;
newDefect.castingDetails = casting;
printf("Enter Defect Type (shrinkage/porosity): ");
scanf("%s", defectType);
strcpy(newDefect.defectType, defectType);
if (strcmp(defectType, "shrinkage") == 0)
{
    printf("Enter Shrinkage Ratio: ");
    scanf("%f", &defectValue);
    newDefect.defect.shrinkage.shrinkageRatio = defectValue;
}

```

```

    }
    else if (strcmp(defectType, "porosity") == 0)
    {
        printf("Enter Porosity Percentage: ");
        scanf("%f", &defectValue);
        newDefect.defect.porosity.porosityPercentage = defectValue;
    }
    else
    {
        printf("Invalid defect type\n");
        return;
    }
    *defects = (struct DefectRecord *)realloc(*defects, (*defectCount + 1) *
                                                sizeof(struct DefectRecord));
    (*defects)[*defectCount] = newDefect;
    (*defectCount)++;
    printf("Defect record added successfully\n");
}

void displayDefectRecords(struct DefectRecord *defects, int defectCount)
{
    if (defectCount == 0)
    {
        printf("No defect records available\n");
        return;
    }
    printf("\nDefect Records\n");

```

```

for (int i = 0; i < defectCount; i++)
{
    printf("\nCasting ID: %d\n", *(defects[i].castingDetails->castingID));
    printf("Material: %s\n", defects[i].castingDetails->material);
    printf("Dimensions: %.2f x %.2f x %.2f\n",
           defects[i].castingDetails->dimensions[0],
           defects[i].castingDetails->dimensions[1],
           defects[i].castingDetails->dimensions[2]);
    printf("Defect Type: %s\n", defects[i].defectType);
    if (strcmp(defects[i].defectType, "shrinkage") == 0)
        printf("Shrinkage Ratio: %.2f\n",
               defects[i].defect.shrinkage.shrinkageRatio);
    else if (strcmp(defects[i].defectType, "porosity") == 0)
        printf("Porosity Percentage: %.2f%%\n",
               defects[i].defect.porosity.porosityPercentage);
}
}

```

13. Metallurgical Lab Automation

Description:

Automate a metallurgical lab using structures for sample details, arrays for test results, and strings for equipment names.

Specifications:

- Structure: Contains sample ID, type, and dimensions.
- Array: Test results.
- Strings: Equipment names.
- const Pointers: Protect sample details.

- Double Pointers: Allocate and manage dynamic test records.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct SampleDetails
```

```
{
```

```
    const int *sampleID;
```

```
    const char *sampleType;
```

```
    float dimensions[3];
```

```
};
```

```
struct TestRecord
```

```
{
```

```
    const struct SampleDetails *sampleDetails;
```

```
    float *testResults;
```

```
    int testCount;
```

```
    char **equipmentNames;
```

```
};
```

```
void addTestRecord(struct TestRecord **records, int *count);
```

```
void displayTestRecords(struct TestRecord *records, int count);
```

```
void freeTestRecords(struct TestRecord *records, int count);
```

```
int main()
```

```
{
```

```

struct TestRecord *records = NULL;
int recordCount = 0, option;
do
{
    printf("\nMetallurgical Lab Automation\n");
    printf("1. Add Test Record\n");
    printf("2. Display Test Records\n");
    printf("3. Exit\n");
    printf("Enter the option: ");
    scanf("%d", &option);
    switch (option)
    {
        case 1: addTestRecord(&records, &recordCount);
                break;
        case 2: displayTestRecords(records, recordCount);
                break;
        case 3: freeTestRecords(records, recordCount);
                printf("Exiting\n");
                break;
        default: printf("Invalid option\n");
    }
} while (option != 3);
return 0;
}

void addTestRecord(struct TestRecord **records, int *count)
{

```

```

struct SampleDetails *sample = (struct SampleDetails *)malloc(sizeof(struct
                                                                    SampleDetails));

struct TestRecord newRecord;
int sampleID, testCount;
char sampleType[50], equipmentName[50];
printf("Enter Sample ID: ");
scanf("%d", &sampleID);
int *sampleIDPtr = (int *)malloc(sizeof(int));
*sampleIDPtr = sampleID;
sample->sampleID = sampleIDPtr;
printf("Enter Sample Type: ");
scanf(" %[^\\n]", sampleType);
sample->sampleType = (char *)malloc((strlen(sampleType) + 1) *
                                                                    sizeof(char));
strcpy((char *)sample->sampleType, sampleType);
printf("Enter Dimensions (Length, Width, Height): ");
scanf("%f %f %f", &sample->dimensions[0], &sample->dimensions[1],
                                                                    &sample->dimensions[2]);

newRecord.sampleDetails = sample;
printf("Enter number of test results: ");
scanf("%d", &testCount);
newRecord.testCount = testCount;
newRecord.testResults = (float *)malloc(testCount * sizeof(float));
printf("Enter Test Results:\\n");
for (int i = 0; i < testCount; i++)
{
    printf("Test Result %d: ", i + 1);

```

```

        scanf("%f", &newRecord.testResults[i]);
    }
    newRecord.equipmentNames = (char **)malloc(testCount * sizeof(char *));
    for (int i = 0; i < testCount; i++)
    {
        printf("Enter Equipment Name for Test %d: ", i + 1);
        scanf(" %[^\\n]", equipmentName);
        newRecord.equipmentNames[i] = (char *)malloc((strlen(equipmentName)
                                                         + 1) * sizeof(char));
        strcpy(newRecord.equipmentNames[i], equipmentName);
    }
    *records = (struct TestRecord *)realloc(*records, (*count + 1) * sizeof(struct
                                                         TestRecord));

    (*records)[*count] = newRecord;
    (*count)++;
    printf("Test record added successfully\\n");
}

```

```

void displayTestRecords(struct TestRecord *records, int count)
{
    if (count == 0){
        printf("No test records available\\n");
        return;
    }
    printf("\\nTest Records:\\n");
    for (int i = 0; i < count; i++){
        printf("\\nSample ID: %d\\n", *(records[i].sampleDetails->sampleID));
    }
}

```

```

printf("Sample Type: %s\n", records[i].sampleDetails->sampleType);
printf("Dimensions: %.2f x %.2f x %.2f\n",
        records[i].sampleDetails->dimensions[0],
        records[i].sampleDetails->dimensions[1],
        records[i].sampleDetails->dimensions[2]);

printf("Test Results:\n");
for (int j = 0; j < records[i].testCount; j++)
{
    printf("Test Result %d: %.2f\n", j + 1, records[i].testResults[j]);
    printf("Equipment Used: %s\n", records[i].equipmentNames[j]);
}
}
}

```

```

void freeTestRecords(struct TestRecord *records, int count)
{
    for (int i = 0; i < count; i++)
    {
        free((int *)records[i].sampleDetails->sampleID);
        free((char *)records[i].sampleDetails->sampleType);
        free(records[i].sampleDetails);
        free(records[i].testResults);
        for (int j = 0; j < records[i].testCount; j++)
            free(records[i].equipmentNames[j]);
        free(records[i].equipmentNames);
    }
    free(records);
}

```

14. Metal Hardness Testing System

Description:

Develop a program to track metal hardness tests using structures for test data, arrays for hardness values, and unions for variable hardness scales.

Specifications:

- Structure: Stores test ID, method, and result.
- Union: Represents variable hardness scales (Rockwell or Brinell).
- Array: Hardness values.
- const Pointers: Protect test data.
- Double Pointers: Dynamic hardness record allocation.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
union HardnessScale
```

```
{
```

```
    float rockwellValue;
```

```
    float brinellValue;
```

```
};
```

```
struct HardnessTestData
```

```
{
```

```
    const int *testID;
```

```
    const char *method;
```

```
    const char *result;
```

```
};
```

```

struct TestRecord
{
    const struct HardnessTestData *testData;
    union HardnessScale *hardnessValues;
    int valueCount;
};

void addTestRecord(struct TestRecord **records, int *count);
void displayTestRecords(struct TestRecord *records, int count);
void freeTestRecords(struct TestRecord *records, int count);

int main()
{
    struct TestRecord *records = NULL;
    int recordCount = 0, option;

    do
    {
        printf("\nMetal Hardness Testing System\n");
        printf("1. Add Hardness Test Record\n");
        printf("2. Display Hardness Test Records\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addTestRecord(&records, &recordCount);

```

```

        break;
    case 2: displayTestRecords(records, recordCount);
        break;
    case 3: freeTestRecords(records, recordCount);
        printf("Exiting the system\n");
        break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addTestRecord(struct TestRecord **records, int *count)
{
    struct HardnessTestData *testData = (struct HardnessTestData
                                         *)malloc(sizeof(struct HardnessTestData));

    struct TestRecord newRecord;
    int testID, valueCount;
    char method[50], result[50];
    char scaleType[20];
    printf("Enter Test ID: ");
    scanf("%d", &testID);
    int *testIDPtr = (int *)malloc(sizeof(int));
    *testIDPtr = testID;
    testData->testID = testIDPtr;
    printf("Enter Test Method (Rockwell/Brinell): ");
    scanf(" %[^\n]", method);
}

```

```

testData->method = (char *)malloc((strlen(method) + 1) * sizeof(char));
strcpy((char *)testData->method, method);
printf("Enter Test Result (e.g., Passed, Failed): ");
scanf(" %[^\\n]", result);
testData->result = (char *)malloc((strlen(result) + 1) * sizeof(char));
strcpy((char *)testData->result, result);
newRecord.testData = testData;
printf("Enter number of hardness values: ");
scanf("%d", &valueCount);
newRecord.valueCount = valueCount;
newRecord.hardnessValues = (union HardnessScale *)malloc(valueCount *
                                                             sizeof(union HardnessScale));
printf("Enter Hardness Values:\\n");
for (int i = 0; i < valueCount; i++)
{
    printf("Hardness Value %d: ", i + 1);
    if (strcmp(method, "Rockwell") == 0)
    {
        printf("Enter Rockwell Value: ");
        scanf("%f", &newRecord.hardnessValues[i].rockwellValue);
    }
    else if (strcmp(method, "Brinell") == 0)
    {
        printf("Enter Brinell Value: ");
        scanf("%f", &newRecord.hardnessValues[i].brinellValue);
    }
    else

```

```

    {
        printf("Invalid test method\n");
        free(newRecord.hardnessValues);
        return;
    }
}

*records = (struct TestRecord *)realloc(*records, (*count + 1) * sizeof(struct
                                     TestRecord));

(*records)[*count] = newRecord;
(*count)++;
printf("Test record added successfully\n");
}

```

```

void displayTestRecords(struct TestRecord *records, int count)
{
    if (count == 0)
    {
        printf("No test records available\n");
        return;
    }
    printf("\nHardness Test Records:\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nTest ID: %d\n", *(records[i].testData->testID));
        printf("Method: %s\n", records[i].testData->method);
        printf("Result: %s\n", records[i].testData->result);
        printf("Hardness Values:\n");
    }
}

```

```

for (int j = 0; j < records[i].valueCount; j++)
{
    if (strcmp(records[i].testData->method, "Rockwell") == 0)
        printf("Rockwell Value %d: %.2f\n", j + 1,
                records[i].hardnessValues[j].rockwellValue);
    else if (strcmp(records[i].testData->method, "Brinell") == 0)
        printf("Brinell Value %d: %.2f\n", j + 1,
                records[i].hardnessValues[j].brinellValue);
}
}
}

```

```

void freeTestRecords(struct TestRecord *records, int count)
{
    for (int i = 0; i < count; i++)
    {
        free((int *)records[i].testData->testID);
        free((char *)records[i].testData->method);
        free((char *)records[i].testData->result);
        free((void *)records[i].testData);
        free(records[i].hardnessValues);
    }
    free(records);
}

```

15. Powder Metallurgy Process Tracker

Description:

Create a program to track powder metallurgy processes using structures for material details, arrays for particle size distribution, and unions for variable powder properties.

Specifications:

- Structure: Contains material ID, type, and density.
- Union: Represents powder properties.
- Array: Particle size distribution data.
- const Pointers: Protect material configurations.
- Double Pointers: Allocate and manage powder data.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
union PowderProperties
```

```
{
```

```
    float flowability;
```

```
    float porosity;
```

```
    char particleShape[50];
```

```
};
```

```
struct MaterialDetails
```

```
{
```

```
    const int *materialID;
```

```
    const char *materialType;
```

```
    float materialDensity;
```

```
};
```

```
struct PowderSample
```

```
{  
    const struct MaterialDetails *materialData;  
    union PowderProperties *powderProperties;  
    float *particleSizeDistribution;  
    int sizeCount;  
};
```

```
void addPowderSample(struct PowderSample **samples, int *count);
```

```
void displayPowderSamples(struct PowderSample *samples, int count);
```

```
void freePowderSamples(struct PowderSample *samples, int count);
```

```
int main()
```

```
{  
    struct PowderSample *samples = NULL;  
    int sampleCount = 0, option;  
    do  
    {  
        printf("\nPowder Metallurgy Process Tracker\n");  
        printf("1. Add Powder Sample\n");  
        printf("2. Display Powder Samples\n");  
        printf("3. Exit\n");  
        printf("Enter the option: ");  
        scanf("%d", &option);  
        switch (option)
```

```

{
    case 1: addPowderSample(&samples, &sampleCount);
        break;
    case 2: displayPowderSamples(samples, sampleCount);
        break;
    case 3: freePowderSamples(samples, sampleCount);
        printf("Exiting\n");
        break;
    default:printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addPowderSample(struct PowderSample **samples, int *count)
{
    struct MaterialDetails *material = (struct MaterialDetails
                                        *)malloc(sizeof(struct MaterialDetails));

    struct PowderSample newSample;
    int materialID, sizeCount;
    char materialType[50], shape[50];
    printf("Enter Material ID: ");
    scanf("%d", &materialID);
    int *materialIDPtr = (int *)malloc(sizeof(int));
    *materialIDPtr = materialID;
    material->materialID = materialIDPtr;
    printf("Enter Material Type: ");

```

```

scanf(" %[^\\n]", materialType);
material->materialType = (char *)malloc((strlen(materialType) + 1) *
                                         sizeof(char));
strcpy((char *)material->materialType, materialType);
printf("Enter Material Density (g/cm^3): ");
scanf("%f", &material->materialDensity);
newSample.materialData = material;
newSample.powderProperties = (union PowderProperties *)
                             malloc(sizeof(union PowderProperties));
printf("Enter Powder Property Type (flowability, porosity, or particle shape):
                                             ");

scanf(" %[^\\n]", shape);
if (strcmp(shape, "flowability") == 0)
{
    printf("Enter Flowability: ");
    scanf("%f", &newSample.powderProperties->flowability);
}
else if (strcmp(shape, "porosity") == 0)
{
    printf("Enter Porosity: ");
    scanf("%f", &newSample.powderProperties->porosity);
}
else if (strcmp(shape, "particle shape") == 0)
{
    printf("Enter Particle Shape (e.g., spherical, irregular): ");
    scanf(" %[^\\n]", newSample.powderProperties->particleShape);
}

```

```

else
{
    printf("Invalid powder property type.\n");
    free(newSample.powderProperties);
    return;
}
printf("Enter number of particle sizes: ");
scanf("%d", &sizeCount);
newSample.sizeCount = sizeCount;
newSample.particleSizeDistribution = (float *)malloc(sizeCount *
                                                    sizeof(float));
printf("Enter Particle Size Distribution (microns):\n");
for (int i = 0; i < sizeCount; i++)
{
    printf("Particle Size %d (microns): ", i + 1);
    scanf("%f", &newSample.particleSizeDistribution[i]);
}
*samples = (struct PowderSample *)realloc(*samples, (*count + 1) *
                                           sizeof(struct PowderSample));
(*samples)[*count] = newSample;
(*count)++;
printf("Powder sample added successfully\n");
}

void displayPowderSamples(struct PowderSample *samples, int count)
{
    if (count == 0)

```

```

{
    printf("No powder samples available\n");
    return;
}
printf("\nPowder Samples:\n");
for (int i = 0; i < count; i++)
{
    printf("\nMaterial ID: %d\n", *(samples[i].materialData->materialID));
    printf("Material Type: %s\n", samples[i].materialData->materialType);
    printf("Material Density: %.2f g/cm^3\n",
           samples[i].materialData->materialDensity);
    printf("Powder Properties:\n");
    if (samples[i].powderProperties->flowability > 0)
        printf("Flowability: %.2f mm/s\n",
               samples[i].powderProperties->flowability);
    else if (samples[i].powderProperties->porosity > 0)
        printf("Porosity: %.2f %%\n", samples[i].powderProperties->porosity);
    else
        printf("Particle Shape: %s\n",
               samples[i].powderProperties->particleShape);
    printf("Particle Size Distribution (microns):\n");
    for (int j = 0; j < samples[i].sizeCount; j++)
        printf("Particle Size %d: %.2f microns\n", j + 1,
               samples[i].particleSizeDistribution[j]);
    }
}

```

```

void freePowderSamples(struct PowderSample *samples, int count)
{
    for (int i = 0; i < count; i++)
    {
        free((int *)samples[i].materialData->materialID);
        free((char *)samples[i].materialData->materialType);
        free((void *)samples[i].materialData);
        free(samples[i].powderProperties);
        free(samples[i].particleSizeDistribution);
    }
    free(samples);
}

```

16. Metal Recycling Analysis

Description:

Develop a program to analyze recycled metal data using structures for material details, arrays for impurity levels, and strings for recycling methods.

Specifications:

- Structure: Holds material ID, type, and recycling method.
- Array: Impurity levels.
- Strings: Recycling methods.
- const Pointers: Protect material details.
- Double Pointers: Allocate dynamic recycling records.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct MaterialDetails
{
    const int *materialID;
    const char *materialType;
    const char *recyclingMethod;
};
```

```
struct RecyclingRecord
{
    const struct MaterialDetails *materialData;
    float *impurityLevels;
    int impurityCount;
};
```

```
void addRecyclingRecord(struct RecyclingRecord **records, int *count);
void displayRecyclingRecords(struct RecyclingRecord *records, int count);
void freeRecyclingRecords(struct RecyclingRecord *records, int count);
```

```
int main()
{
    struct RecyclingRecord *records = NULL;
    int recordCount = 0, option;
    do
    {
        printf("\nMetal Recycling Analysis\n");
        printf("1. Add Recycling Record\n");
        printf("2. Display Recycling Records\n");
```

```

printf("3. Exit\n");
printf("Enter the option: ");
scanf("%d", &option);
switch (option)
{
    case 1: addRecyclingRecord(&records, &recordCount);
            break;
    case 2: displayRecyclingRecords(records, recordCount);
            break;
    case 3: freeRecyclingRecords(records, recordCount);
            printf("Exiting\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

void addRecyclingRecord(struct RecyclingRecord **records, int *count)
{
    struct MaterialDetails *material = (struct MaterialDetails *)
                                     malloc(sizeof(struct MaterialDetails));
    struct RecyclingRecord newRecord;
    int materialID, impurityCount;
    char materialType[50], recyclingMethod[50];
    printf("Enter Material ID: ");
    scanf("%d", &materialID);

```

```

int *materialIDPtr = (int *)malloc(sizeof(int));
*materialIDPtr = materialID;
material->materialID = materialIDPtr;
printf("Enter Material Type: ");
scanf(" %[^\\n]", materialType);
material->materialType = (char *)malloc((strlen(materialType) + 1) *
                                         sizeof(char));
strcpy((char *)material->materialType, materialType);
printf("Enter Recycling Method: ");
scanf(" %[^\\n]", recyclingMethod);
material->recyclingMethod = (char *)malloc((strlen(recyclingMethod) + 1) *
                                         sizeof(char));
strcpy((char *)material->recyclingMethod, recyclingMethod);
newRecord.materialData = material;
printf("Enter number of impurity levels: ");
scanf("%d", &impurityCount);
newRecord.impurityCount = impurityCount;
newRecord.impurityLevels = (float *)malloc(impurityCount * sizeof(float));
printf("Enter Impurity Levels (percentage):\\n");
for (int i = 0; i < impurityCount; i++)
{
    printf("Impurity Level %d: ", i + 1);
    scanf("%f", &newRecord.impurityLevels[i]);
}
*records = (struct RecyclingRecord *)realloc(*records, (*count + 1) *
                                             sizeof(struct RecyclingRecord));
(*records)[*count] = newRecord;

```

```

    (*count)++;
    printf("Recycling record added successfully\n");
}

void displayRecyclingRecords(struct RecyclingRecord *records, int count)
{
    if (count == 0)
    {
        printf("No recycling records available\n");
        return;
    }
    printf("\nRecycling Records:\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nMaterial ID: %d\n", *(records[i].materialData->materialID));
        printf("Material Type: %s\n", records[i].materialData->materialType);
        printf("Recycling Method: %s\n", \
                records[i].materialData->recyclingMethod);
        printf("Impurity Levels (percentage):\n");
        for (int j = 0; j < records[i].impurityCount; j++)
            printf("Impurity Level %d: %.2f%%\n", j + 1,
                    records[i].impurityLevels[j]);
    }
}

void freeRecyclingRecords(struct RecyclingRecord *records, int count)
{

```

```

for (int i = 0; i < count; i++)
{
    free((int *)records[i].materialData->materialID);
    free((char *)records[i].materialData->materialType);
    free((char *)records[i].materialData->recyclingMethod);
    free((void *)records[i].materialData);
    free(records[i].impurityLevels);
}
free(records);
}

```

17. Rolling Mill Performance Tracker

Description:

Design a system to track rolling mill performance using structures for mill configurations, arrays for output data, and strings for material types.

Specifications:

- Structure: Stores mill ID, roll diameter, and speed.
- Array: Output data.
- Strings: Material types.
- const Pointers: Protect mill configurations.
- Double Pointers: Manage rolling mill records dynamically.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct MillConfig
```

```
{  
    const int *millID;  
    float rollDiameter;  
    float speed;  
};
```

```
struct RollingMillRecord  
{  
    const struct MillConfig *millConfig;  
    float *outputData;  
    int outputCount;  
    const char *materialType;  
};
```

```
void addRollingMillRecord(struct RollingMillRecord **records, int *count);  
void displayRollingMillRecords(struct RollingMillRecord *records, int count);  
void freeRollingMillRecords(struct RollingMillRecord *records, int count);
```

```
int main()  
{  
    struct RollingMillRecord *records = NULL;  
    int recordCount = 0, option;  
    do  
    {  
        printf("\nRolling Mill Performance Tracker\n");  
        printf("1. Add Rolling Mill Record\n");  
        printf("2. Display Rolling Mill Records\n");
```

```

    printf("3. Exit\n");
    printf("Enter the option: ");
    scanf("%d", &option);
    switch (option)
    {
        case 1: addRollingMillRecord(&records, &recordCount);
                break;
        case 2: displayRollingMillRecords(records, recordCount);
                break;
        case 3: freeRollingMillRecords(records, recordCount);
                printf("Exiting\n");
                break;
        default: printf("Invalid option\n");
    }
} while (option != 3);
return 0;
}

void addRollingMillRecord(struct RollingMillRecord **records, int *count)
{
    struct MillConfig *millConfig = (struct MillConfig *)malloc
                                    (sizeof(struct MillConfig));

    struct RollingMillRecord newRecord;
    int millID, outputCount;
    char materialType[50];
    printf("Enter Mill ID: ");
    scanf("%d", &millID);

```

```

int *millIDPtr = (int *)malloc(sizeof(int));
*millIDPtr = millID;
millConfig->millID = millIDPtr;
printf("Enter Roll Diameter: ");
scanf("%f", &millConfig->rollDiameter);
printf("Enter Mill Speed: ");
scanf("%f", &millConfig->speed);
newRecord.millConfig = millConfig;
printf("Enter Material Type: ");
scanf(" %[^\\n]", materialType);
newRecord.materialType = (char *)malloc((strlen(materialType) + 1) *
                                         sizeof(char));
strcpy((char *)newRecord.materialType, materialType);
printf("Enter number of output data points: ");
scanf("%d", &outputCount);
newRecord.outputCount = outputCount;
newRecord.outputData = (float *)malloc(outputCount * sizeof(float));
printf("Enter Output Data (e.g., thickness, yield):\\n");
for (int i = 0; i < outputCount; i++)
{
    printf("Output Data %d: ", i + 1);
    scanf("%f", &newRecord.outputData[i]);
}
*records = (struct RollingMillRecord *)realloc(*records, (*count + 1) *
                                                sizeof(struct RollingMillRecord));
(*records)[*count] = newRecord;
(*count)++;

```

```

    printf("Rolling mill record added successfully\n");
}

void displayRollingMillRecords(struct RollingMillRecord *records, int count)
{
    if (count == 0)
    {
        printf("No rolling mill records available\n");
        return;
    }
    printf("\nRolling Mill Records:\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nMill ID: %d\n", *(records[i].millConfig->millID));
        printf("Roll Diameter: %.2f cm\n", records[i].millConfig->rollDiameter);
        printf("Mill Speed: %.2f m/min\n", records[i].millConfig->speed);
        printf("Material Type: %s\n", records[i].materialType);
        printf("Output Data:\n");
        for (int j = 0; j < records[i].outputCount; j++)
            printf("Output Data %d: %.2f\n", j + 1, records[i].outputData[j]);
    }
}

void freeRollingMillRecords(struct RollingMillRecord *records, int count)
{
    for (int i = 0; i < count; i++)
    {

```

```

        free((int *)records[i].millConfig->millID);
        free((void *)records[i].millConfig);
        free((char *)records[i].materialType);
        free(records[i].outputData);
    }
    free(records);
}

```

18. Thermal Expansion Analysis

Description:

Create a program to analyze thermal expansion using arrays for temperature data, structures for material properties, and unions for variable coefficients.

Specifications:

- Structure: Contains material ID, type, and expansion coefficient.
- Union: Represents variable coefficients.
- Array: Temperature data.
- const Pointers: Protect material properties.
- Double Pointers: Dynamic thermal expansion record allocation.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

union ExpansionCoefficient
{
    float linearCoefficient;
    float volumetricCoefficient;
}

```

```
};
```

```
struct MaterialProperties
```

```
{
```

```
    const int *materialID;
```

```
    const char *materialType;
```

```
    union ExpansionCoefficient expansionCoefficient;
```

```
};
```

```
struct ThermalExpansionRecord
```

```
{
```

```
    const struct MaterialProperties *materialData;
```

```
    float *temperatureData;
```

```
    int temperatureCount;
```

```
};
```

```
void addThermalExpansionRecord(struct ThermalExpansionRecord **records,  
                               int *count);
```

```
void displayThermalExpansionRecords(struct ThermalExpansionRecord  
                                    *records, int count);
```

```
void freeThermalExpansionRecords(struct ThermalExpansionRecord *records,  
                                  int count);
```

```
int main()
```

```
{
```

```
    struct ThermalExpansionRecord *records = NULL;
```

```
    int recordCount = 0, option;
```

```

do
{
    printf("\nThermal Expansion Analysis\n");
    printf("1. Add Thermal Expansion Record\n");
    printf("2. Display Thermal Expansion Records\n");
    printf("3. Exit\n");
    printf("Enter the option: ");
    scanf("%d", &option);
    switch (option)
    {
        case 1: addThermalExpansionRecord(&records, &recordCount);
                break;
        case 2: displayThermalExpansionRecords(records, recordCount);
                break;
        case 3: freeThermalExpansionRecords(records, recordCount);
                printf("Exiting\n");
                break;
        default: printf("Invalid option\n");
    }
} while (option != 3);
return 0;
}

void addThermalExpansionRecord(struct ThermalExpansionRecord **records,
                              int *count)
{
    struct MaterialProperties *material = (struct MaterialProperties *)

```

```

        malloc(sizeof(struct MaterialProperties));

struct ThermalExpansionRecord newRecord;

int materialID, temperatureCount;

char materialType[50];

float linearCoefficient, volumetricCoefficient;

printf("Enter Material ID: ");

scanf("%d", &materialID);

int *materialIDPtr = (int *)malloc(sizeof(int));

*materialIDPtr = materialID;

material->materialID = materialIDPtr;

printf("Enter Material Type: ");

scanf(" %[^\\n]", materialType);

material->materialType = (char *)malloc((strlen(materialType) + 1) *
                                         sizeof(char));

strcpy((char *)material->materialType, materialType);

printf("Enter 1 for Linear Expansion Coefficient or 2 for Volumetric Expansion
                                         Coefficient: ");

int choice;

scanf("%d", &choice);

if (choice == 1)
{
    printf("Enter Linear Expansion Coefficient: ");

    scanf("%f", &linearCoefficient);

    material->expansionCoefficient.linearCoefficient = linearCoefficient;
}

else if (choice == 2)
{

```

```

    printf("Enter Volumetric Expansion Coefficient: ");
    scanf("%f", &volumetricCoefficient);
    material->expansionCoefficient.volumetricCoefficient =
                                                    volumetricCoefficient;
}
else
    printf("Invalid choice\n");
newRecord.materialData = material;
printf("Enter number of temperature data points: ");
scanf("%d", &temperatureCount);
newRecord.temperatureCount = temperatureCount;
newRecord.temperatureData = (float *)malloc(temperatureCount *
                                                    sizeof(float));

printf("Enter Temperature Data:\n");
for (int i = 0; i < temperatureCount; i++)
{
    printf("Temperature %d: ", i + 1);
    scanf("%f", &newRecord.temperatureData[i]);
}
*records = (struct ThermalExpansionRecord *)realloc(*records, (*count + 1)
                                                    * sizeof(struct ThermalExpansionRecord));
(*records)[*count] = newRecord;
(*count)++;
printf("Thermal expansion record added successfully\n");
}

void displayThermalExpansionRecords(struct ThermalExpansionRecord

```

*records, int count)

```
{
    if (count == 0)
    {
        printf("No thermal expansion records available\n");
        return;
    }
    printf("\nThermal Expansion Records:\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nMaterial ID: %d\n", *(records[i].materialData->materialID));
        printf("Material Type: %s\n", records[i].materialData->materialType);
        if (records[i].materialData->expansionCoefficient.linearCoefficient != 0)
            printf("Linear Expansion Coefficient: %f\n",
                records[i].materialData->expansionCoefficient.linearCoefficient);
        else
            printf("Volumetric Expansion Coefficient: %f\n",
                records[i].materialData->expansionCoefficient.volumetricCoefficient);
        printf("Temperature Data:\n");
        for (int j = 0; j < records[i].temperatureCount; j++)
            printf("Temperature %d: %.2f°C\n", j + 1, records[i].temperatureData[j]);
    }
}
```

void freeThermalExpansionRecords(struct ThermalExpansionRecord *records,
int count)

```
{
    for (int i = 0; i < count; i++)
```

```

{
    free((int *)records[i].materialData->materialID);
    free((void *)records[i].materialData->materialType);
    free((void *)records[i].materialData);
    free(records[i].temperatureData);
}
free(records);
}

```

19. Metal Melting Point Analyzer

Description:

Develop a program to analyze melting points using structures for metal details, arrays for temperature data, and strings for metal names.

Specifications:

- Structure: Stores metal ID, name, and melting point.
- Array: Temperature data.
- Strings: Metal names.
- const Pointers: Protect metal details.
- Double Pointers: Allocate dynamic melting point records.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct MetalDetails
```

```
{
```

```
    const int *metalID;
```

```

    const char *metalName;
    float meltingPoint;
};

struct MeltingPointRecord
{
    const struct MetalDetails *metalData;
    float *temperatureData;
    int temperatureCount;
};

void addMeltingPointRecord(struct MeltingPointRecord **records, int *count);
void displayMeltingPointRecords(struct MeltingPointRecord *records,
                                int count);
void freeMeltingPointRecords(struct MeltingPointRecord *records, int count);

int main()
{
    struct MeltingPointRecord *records = NULL;
    int recordCount = 0, option;
    do{
        printf("\nMetal Melting Point Analyzer\n");
        printf("1. Add Melting Point Record\n");
        printf("2. Display Melting Point Records\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
    }

```

```

switch (option){
    case 1: addMeltingPointRecord(&records, &recordCount);
        break;
    case 2: displayMeltingPointRecords(records, recordCount);
        break;
    case 3: freeMeltingPointRecords(records, recordCount);
        printf("Exiting\n");
        break;
    default:printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addMeltingPointRecord(struct MeltingPointRecord **records, int *count)
{
    struct MetalDetails *metal = (struct MetalDetails *)malloc
                                (sizeof(struct MetalDetails));

    struct MeltingPointRecord newRecord;
    int metalID, temperatureCount;
    char metalName[50];
    printf("Enter Metal ID: ");
    scanf("%d", &metalID);
    int *metalIDPtr = (int *)malloc(sizeof(int));
    *metalIDPtr = metalID;
    metal->metalID = metalIDPtr;
    printf("Enter Metal Name: ");

```

```

scanf(" %[^\\n]", metalName);
metal->metalName = (char *)malloc((strlen(metalName) + 1) * sizeof(char));
strcpy((char *)metal->metalName, metalName);
printf("Enter Melting Point: ");
scanf("%f", &metal->meltingPoint);
newRecord.metalData = metal;
printf("Enter number of temperature data points: ");
scanf("%d", &temperatureCount);
newRecord.temperatureCount = temperatureCount;
newRecord.temperatureData = (float *)malloc(temperatureCount *
                                                sizeof(float));

printf("Enter Temperature Data (in Celsius):\\n");
for (int i = 0; i < temperatureCount; i++)
{
    printf("Temperature %d: ", i + 1);
    scanf("%f", &newRecord.temperatureData[i]);
}

*records = (struct MeltingPointRecord *)realloc(*records, (*count + 1) *
                                                sizeof(struct MeltingPointRecord));

(*records)[*count] = newRecord;
(*count)++;
printf("Melting point record added successfully\\n");
}

void displayMeltingPointRecords(struct MeltingPointRecord *records, int count)
{
    if (count == 0)

```

```

{
    printf("No melting point records available\n");
    return;
}
printf("\nMelting Point Records\n");
for (int i = 0; i < count; i++)
{
    printf("\nMetal ID: %d\n", *(records[i].metalData->metalID));
    printf("Metal Name: %s\n", records[i].metalData->metalName);
    printf("Melting Point: %.2f\n", records[i].metalData->meltingPoint);
    printf("Temperature Data (in Celsius):\n");
    for (int j = 0; j < records[i].temperatureCount; j++)
        printf("Temperature %d: %.2f°C\n", j + 1, records[i].temperatureData[j]);
    }
}

void freeMeltingPointRecords(struct MeltingPointRecord *records, int count)
{
    for (int i = 0; i < count; i++)
    {
        free((int *)records[i].metalData->metalID);
        free((void *)records[i].metalData->metalName);
        free((void *)records[i].metalData);
        free(records[i].temperatureData);
    }
    free(records);
}

```

20. Smelting Efficiency Analyzer

Description:

Design a system to analyze smelting efficiency using structures for process details, arrays for energy consumption data, and unions for variable process parameters.

Specifications:

- Structure: Contains process ID, ore type, and efficiency.
- Union: Represents process parameters (energy or duration).
- Array: Energy consumption data.
- const Pointers: Protect process configurations.
- Double Pointers: Manage smelting efficiency records dynamically.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
union ProcessParameters
```

```
{  
    float energyConsumed;  
    float processDuration;  
};
```

```
struct ProcessDetails
```

```
{  
    const int *processID;  
    const char *oreType;  
    float efficiency;  
};
```

```

struct SmeltingEfficiencyRecord
{
    const struct ProcessDetails *processData;
    float *energyConsumptionData;
    int dataCount;
    union ProcessParameters processParams;
};

void addSmeltingEfficiencyRecord(struct SmeltingEfficiencyRecord **records,
                                int *count);

void displaySmeltingEfficiencyRecords(struct SmeltingEfficiencyRecord
                                      *records, int count);

void freeSmeltingEfficiencyRecords(struct SmeltingEfficiencyRecord *records,
                                   int count);

int main()
{
    struct SmeltingEfficiencyRecord *records = NULL;
    int recordCount = 0, option;
    do
    {
        printf("\nSmelting Efficiency Analyzer\n");
        printf("1. Add Smelting Efficiency Record\n");
        printf("2. Display Smelting Efficiency Records\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
    }

```

```

switch (option)
{
    case 1: addSmeltingEfficiencyRecord(&records, &recordCount);
            break;
    case 2: displaySmeltingEfficiencyRecords(records, recordCount);
            break;
    case 3: freeSmeltingEfficiencyRecords(records, recordCount);
            printf("Exiting\n");
            break;
    default: printf("Invalid option\n");
}
} while (option != 3);
return 0;
}

```

```

void addSmeltingEfficiencyRecord(struct SmeltingEfficiencyRecord **records,
                                int *count)
{
    struct ProcessDetails *process = (struct ProcessDetails *)malloc
                                    (sizeof(struct ProcessDetails));

    struct SmeltingEfficiencyRecord newRecord;
    int processID, dataCount;
    char oreType[50];
    float efficiency, energy, duration;
    int choice;

    printf("Enter Process ID: ");
    scanf("%d", &processID);
}

```

```

int *processIDPtr = (int *)malloc(sizeof(int));
*processIDPtr = processID;
process->processID = processIDPtr;
printf("Enter Ore Type: ");
scanf(" %[^\\n]", oreType);
process->oreType = (char *)malloc((strlen(oreType) + 1) * sizeof(char));
strcpy((char *)process->oreType, oreType);
printf("Enter Efficiency (percentage): ");
scanf("%f", &efficiency);
process->efficiency = efficiency;
newRecord.processData = process;
printf("Enter 1 for Energy Consumed or 2 for Process Duration: ");
scanf("%d", &choice);
if (choice == 1)
{
    printf("Enter Energy Consumed (in kWh): ");
    scanf("%f", &energy);
    newRecord.processParams.energyConsumed = energy;
}
else if (choice == 2)
{
    printf("Enter Process Duration (in hours): ");
    scanf("%f", &duration);
    newRecord.processParams.processDuration = duration;
}
else
    printf("Invalid option\\n");

```

```

printf("Enter number of energy consumption data points: ");
scanf("%d", &dataCount);
newRecord.dataCount = dataCount;
newRecord.energyConsumptionData = (float *)malloc(dataCount *
                                                    sizeof(float));

printf("Enter Energy Consumption Data (in kWh):\n");
for (int i = 0; i < dataCount; i++)
{
    printf("Energy Consumption %d: ", i + 1);
    scanf("%f", &newRecord.energyConsumptionData[i]);
}
*records = (struct SmeltingEfficiencyRecord *)realloc(*records, (*count + 1)
                                                    * sizeof(struct SmeltingEfficiencyRecord));
(*records)[*count] = newRecord;
(*count)++;
printf("Smelting efficiency record added successfully\n");
}

void displaySmeltingEfficiencyRecords(struct SmeltingEfficiencyRecord
                                     *records, int count)
{
    if (count == 0){
        printf("No smelting efficiency records available\n");
        return;
    }
    printf("\nSmelting Efficiency Records\n");
    for (int i = 0; i < count; i++)

```

```

{
    printf("\nProcess ID: %d\n", *(records[i].processData->processID));
    printf("Ore Type: %s\n", records[i].processData->oreType);
    printf("Efficiency: %.2f%%\n", records[i].processData->efficiency);
    if (records[i].processParams.energyConsumed != 0)
        printf("Energy Consumed: %.2f kWh\n",
            records[i].processParams.energyConsumed);
    else
        printf("Process Duration: %.2f hours\n",
            records[i].processParams.processDuration);
    printf("Energy Consumption Data (in kWh):\n");
    for (int j = 0; j < records[i].dataCount; j++)
        printf("Energy Consumption %d: %.2f kWh\n", j + 1,
            records[i].energyConsumptionData[j]);
}
}

void freeSmeltingEfficiencyRecords(struct SmeltingEfficiencyRecord *records,
int count)
{
    for (int i = 0; i < count; i++){
        free((int *)records[i].processData->processID);
        free((void *)records[i].processData->oreType);
        free((void *)records[i].processData);
        free(records[i].energyConsumptionData);
    }
    free(records);
}

```